

DS5216: Artificial Intelligence (2024/25)

Programming Assignment 01: LSTMs and Transformers

- **Implementations and Architectures**

Here are the summary of the architectures for the LSTM version and the BERT version for the tweet data analysis

LSTM

This is a basic implementation for the LSTM model and here are the steps for the tweet data classification,

Loading Data: The process starts by bringing the training and testing datasets into the environment from storage.

Cleaning Data: The text data undergoes a cleaning process where unwanted elements like HTML tags and non-alphabetical characters are removed. Common words that don't add much meaning (stop words) are also taken out, and all text is converted to lowercase.

Preparing Datasets: The loaded data is organized for training and testing. This involves removing an identification column and making sure only relevant data points are included. The cleaning step is then applied to both the training and testing portions.

Converting Text to Numbers and Standardizing Length: The cleaned text is converted into a numerical format that the model can understand. This involves assigning a unique number to each word. Then, the sequences of numbers representing the text are adjusted to a consistent length by adding padding or cutting off extra parts. This step also counts the total number of unique words encountered.

Creating the AI Model: A model is put together using different layers designed to handle sequence data. One layer converts the word numbers into rich representations, another is specifically for processing sequences and capturing dependencies, and a final layer provides the output for classification.

Setting up the Model for Training: Before training, the model is configured with settings that guide how it learns. This includes choosing how it will adjust its internal values to minimize errors and what measure will be used to check its performance.

Training and Checking Performance: The model is trained using the prepared data. This process is repeated multiple times to get a more reliable understanding of how well it performs. Before each training run, the model's internal values are reset. During training, the model's performance is monitored, and the best version is saved. After training, the model is tested on a separate dataset to see how accurately it makes predictions.

The model is a sequential model, (meaning the layers are stacked one after another). It consists of three main layers:

- **Embedding Layer:** This layer takes the integer sequences representing the tweets and converts each word index into a dense vector of a fixed size (32 in this case). This helps the model learn relationships between words.
- **LSTM Layer:** This is a type of recurrent neural network (RNN) layer that is particularly good at processing sequential data like text. It has internal memory that allows it to consider the context of words in a sequence. The output dimension of this layer is 64.

- **Dense Layer:** This is the final output layer. It's a fully connected layer with a single neuron and a sigmoid activation function. The sigmoid function squashes the output to a value between 0 and 1, which can be interpreted as the probability of the tweet belonging to one of the two classes (0 or 1).

Model: "sequential_8"

Layer (type)	Output Shape	Param #
embedding_8 (Embedding)	(None, 10, 32)	405,120
lstm_8 (LSTM)	(None, 64)	24,832
dense_8 (Dense)	(None, 1)	65

Total params: 430,017 (1.64 MB)
Trainable params: 430,017 (1.64 MB)
Non-trainable params: 0 (0.00 B)
None

Figure 01: LSTM architecture and parameters

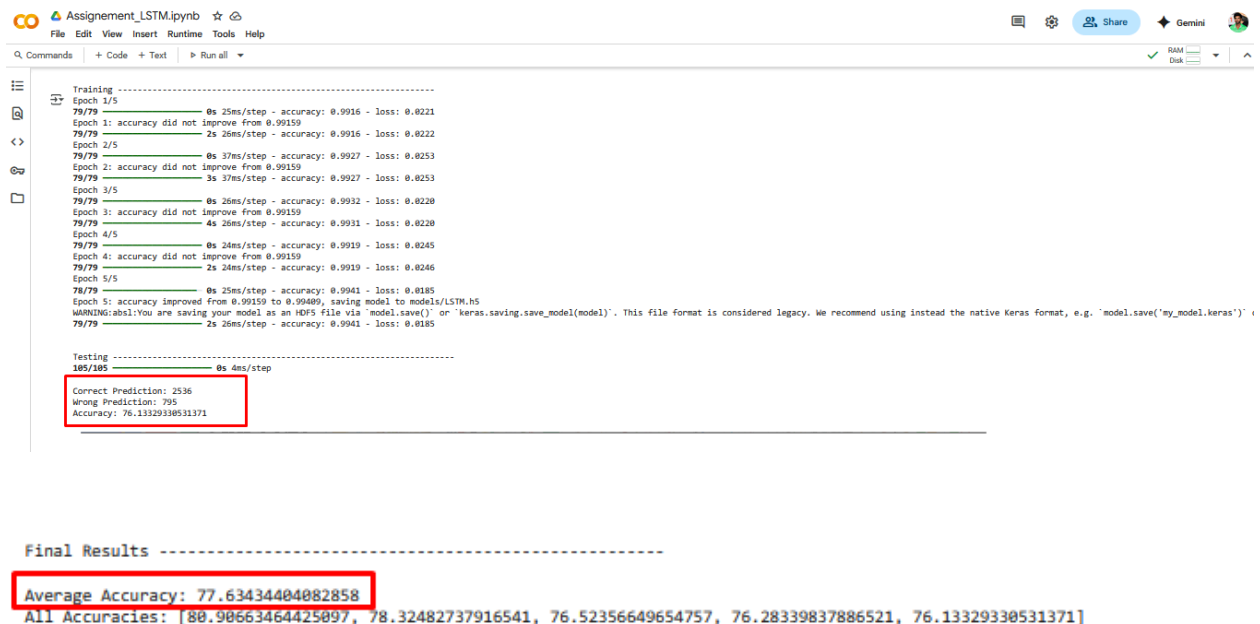


Figure 02: LSTM accuracy

Enhanced LSTM with drop out

Here also the same data processing and model building technique was followed. Here is the summary of the implemented model. Note that for this case some enhancement was done to get the better accuracy than the above LSTM implementation.

The model is a sequential model with the following layers:

- **Embedding layer:** This layer takes the tokenized input and creates dense vector representations of the words. It has 12,660 input dimensions (the total number of unique words) and outputs vectors of 128 dimensions. It also incorporates L2 regularization.

- **LSTM layer:** This is a Long Short-Term Memory layer with 96 units. It processes the sequence of embedded words and captures temporal dependencies. It includes dropout and recurrent dropout for regularization, as well as L2 kernel regularization.
- **Dropout layer:** This layer randomly sets a fraction of the LSTM layer's outputs to zero during training to prevent overfitting.
- **Dense layer:** This is the output layer with a single unit and a sigmoid activation function. It outputs a value between 0 and 1, representing the probability of the input tweet belonging to the positive class

Model: "sequential_10"

Layer (type)	Output Shape	Param #
embedding_10 (Embedding)	(None, 10, 128)	1,620,480
lstm_10 (LSTM)	(None, 96)	86,400
dropout_1 (Dropout)	(None, 96)	0
dense_10 (Dense)	(None, 1)	97

Total params: 1,706,977 (6.51 MB)
Trainable params: 1,706,977 (6.51 MB)
Non-trainable params: 0 (0.00 B)
None

Figure 03: LSTM with dense layer architecture and parameters

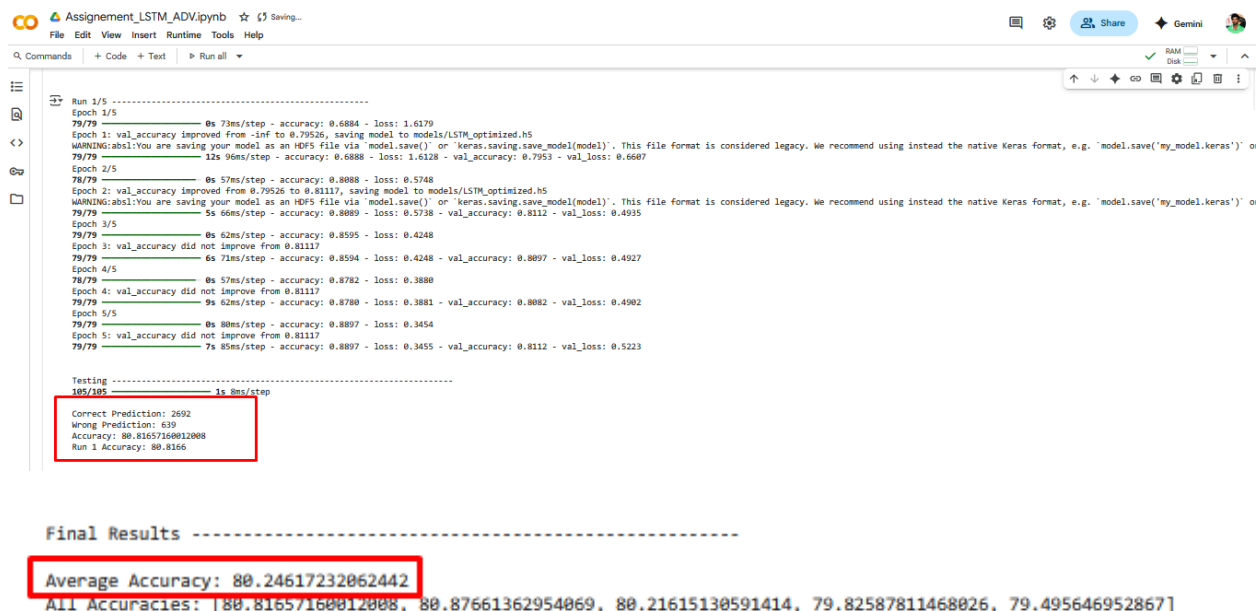


Figure 04: LSTM with dense layer accuracy

Bidirectional LSTM

The same above data proceSSION and model build step followed and here is a summary for the Bidirectional LSTM,

Data loading: Load the training and testing datasets from Google Drive.

Data preprocessing: Clean and preprocess the tweet data, including removing HTML tags, non-alphabetic characters, and stop words, and converting text to lowercase.

Data preparation: Get the preprocessed training and testing data and determine the maximum tweet length.

Tokenization and padding: Tokenize the text data and pad the sequences to a fixed length.

Model building: Construct the Bidirectional LSTM model with specified hyperparameters and regularization.

Model compilation: Compile the model with the Adam optimizer, binary crossentropy loss, and accuracy metric.

Model training and evaluation: Train the model multiple times and evaluate its performance on the test set, reporting the accuracy for each run and the average accuracy.

When considering about the model, it consists of the following layers:

- **Embedding Layer:** Converts input words into dense vector representations.
- **Bidirectional LSTM Layer:** Processes the embedded sequences in both forward and backward directions to capture contextual information. It includes dropout and recurrent dropout for regularization.
- **Dropout Layer:** Further reduces overfitting by randomly setting a fraction of input units to zero.
- **Dense Output Layer:** A single unit with a sigmoid activation function to output the probability of the positive class.

The model is compiled with the Adam optimizer and uses binary crossentropy as the loss function, with accuracy as the evaluation metric. It is trained and evaluated multiple times to assess its performance.

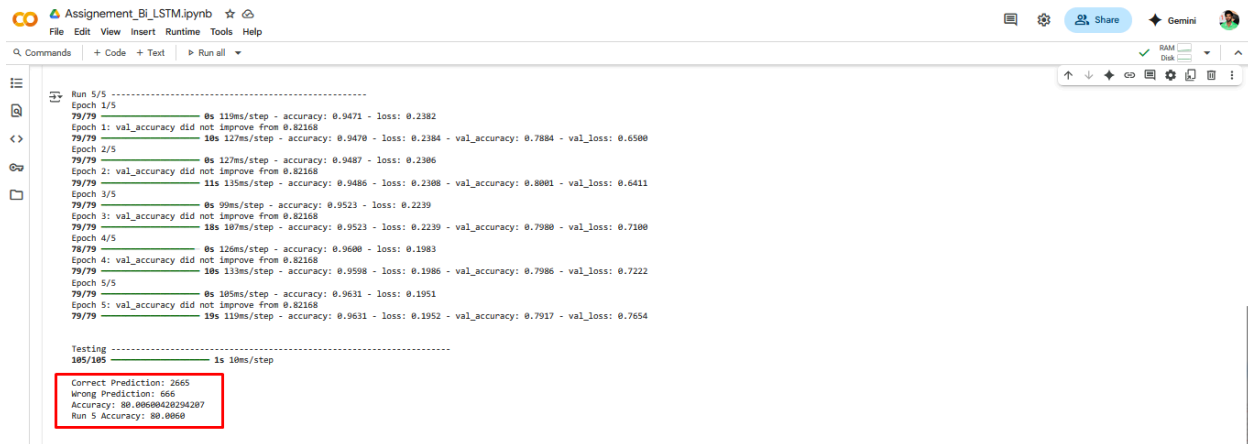
Please refer to the following for the accuracy measurements. Here it was able to observe that the Bidirectional LSTM performed better than the usual LSTM models and was given a good accuracy.

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 10, 128)	1,620,480
bidirectional (Bidirectional)	(None, 192)	172,800
dropout (Dropout)	(None, 192)	0
dense (Dense)	(None, 1)	193

Total params: 1,793,473 (6.84 MB)
Trainable params: 1,793,473 (6.84 MB)
Non-trainable params: 0 (0.00 B)
None

Figure 05: Bidirectional LSTM architecture and parameters



```
Run 5/5 -----
Epoch 1/5
79/79 ----- 0s 110ms/step - accuracy: 0.9471 - loss: 0.2382
Epoch 1: val_accuracy did not improve from 0.82168
79/79 ----- 18s 127ms/step - accuracy: 0.9470 - loss: 0.2384 - val_accuracy: 0.7884 - val_loss: 0.6500
Epoch 2/5
79/79 ----- 0s 127ms/step - accuracy: 0.9487 - loss: 0.2386
Epoch 2: val_accuracy did not improve from 0.82168
79/79 ----- 11s 135ms/step - accuracy: 0.9486 - loss: 0.2308 - val_accuracy: 0.8001 - val_loss: 0.6411
Epoch 3/5
79/79 ----- 0s 99ms/step - accuracy: 0.9523 - loss: 0.2239
Epoch 3: val_accuracy did not improve from 0.82168
79/79 ----- 18s 107ms/step - accuracy: 0.9523 - loss: 0.2239 - val_accuracy: 0.7980 - val_loss: 0.7100
Epoch 4/5
79/79 ----- 0s 126ms/step - accuracy: 0.9600 - loss: 0.1983
Epoch 4: val_accuracy did not improve from 0.82168
79/79 ----- 10s 133ms/step - accuracy: 0.9598 - loss: 0.1986 - val_accuracy: 0.7986 - val_loss: 0.7222
Epoch 5/5
79/79 ----- 0s 105ms/step - accuracy: 0.9631 - loss: 0.1951
Epoch 5: val_accuracy did not improve from 0.82168
79/79 ----- 19s 119ms/step - accuracy: 0.9631 - loss: 0.1952 - val_accuracy: 0.7917 - val_loss: 0.7654

Testing
105/105 ----- 1s 10ms/step
Correct Prediction: 2865
Wrong Prediction: 666
Accuracy: 80.00600420294207
Run 5 Accuracy: 80.0060
```

Final Results -----
Average Accuracy: 80.9306514560192
All Accuracies: [82.13749624737316, 81.95737015911138, 80.90663464425097, 79.64575202641849, 80.00600420294207]

Figure 06: Bidirectional LSTM accuracy

BERT

The BERT model was used to build the predicted model for the same tweet data, here are the steps which are followed to implement the BERT classification model.

Loading and Preprocessing Data: The code first loads your tweet data from CSV files in your Google Drive. It then preprocesses the text by removing HTML tags, non-alphabetical characters, and stopwords, and converts everything to lowercase.

Tokenization: Here it was used the BertTokenizer from the transformers library to convert the processed text into numerical tokens that the BERT model can understand. It also adds padding and attention masks, which are necessary for BERT. The maximum length of the tokenized sequences is determined based on the average tweet length, up to a maximum of 512.

Model build: The build_bert_model function constructs a Keras model using TFBertModel from the transformers library. It takes the tokenized input IDs and attention masks, passes them through the pre-trained BERT model I, and extracts the output corresponding to the [CLS] token. On top of the BERT output, a custom classification head is added. This consists of Dropout layers for regularization and Dense layers with ReLU and Sigmoid activation functions to output a single value between 0 and 1, representing the probability of the positive class.

Model Compilation: The model is compiled with the Adam optimizer and binary cross-entropy loss, which is suitable for binary classification problems. Accuracy is used as the evaluation metric.

Repeated Evaluation: The custom function (please refer to the code) allows for training and evaluating the model multiple times with different random weight initializations to get a more robust estimate of performance. It includes ModelCheckpoint and EarlyStopping callbacks to save the best model and prevent overfitting.

Please refer to the following for the BERT model details and accuracy measure on tweet data,

Model: "functional_2"

Layer (type)	Output Shape	Param #	Connected to
input_ids (InputLayer)	(None, 10)	0	-
attention_mask (InputLayer)	(None, 10)	0	-
bert_layer_2 (BertLayer)	(None, 768)	0	input_ids[0][0], attention_mask[0..
dropout_4 (Dropout)	(None, 768)	0	bert_layer_2[0][..
dense_4 (Dense)	(None, 128)	98,432	dropout_4[0][0]
dropout_5 (Dropout)	(None, 128)	0	dense_4[0][0]
dense_5 (Dense)	(None, 1)	129	dropout_5[0][0]

Total params: 98,561 (385.00 KB)
 Trainable params: 98,561 (385.00 KB)
 Non-trainable params: 0 (0.00 B)
 None

Figure 07: BERT architecture and parameters

32 batch size and 5 epochs

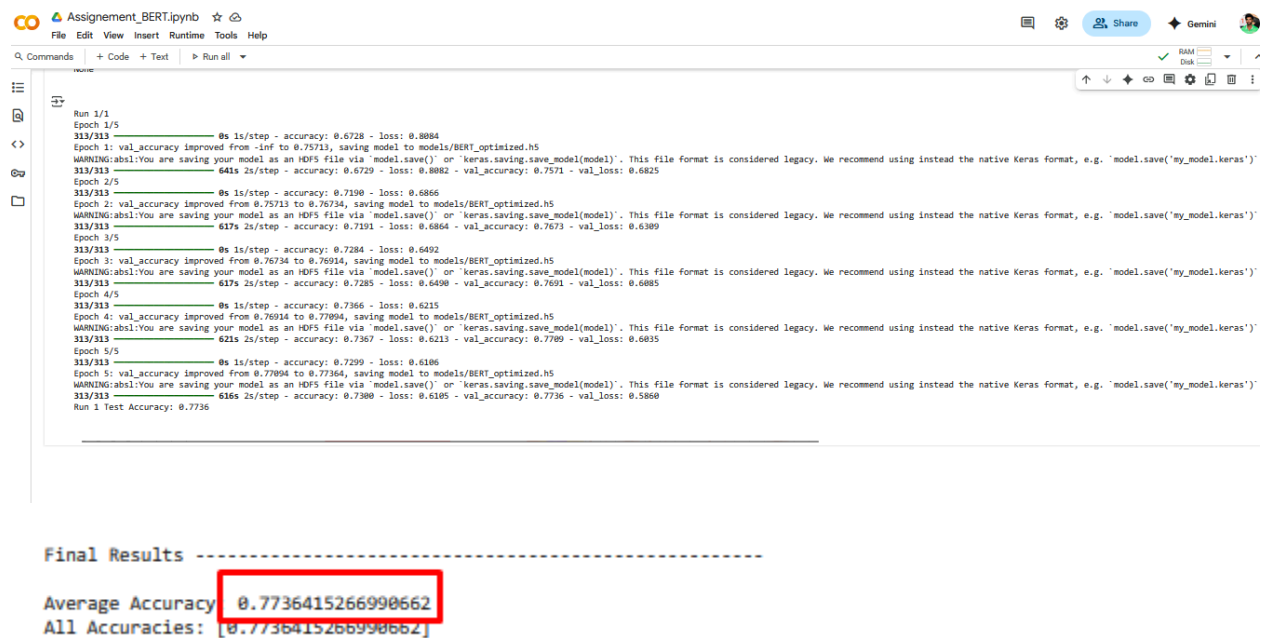


Figure 08: BERT accuracy for 5 epochs

The same BERT model with 64 batch size and 10 epochs

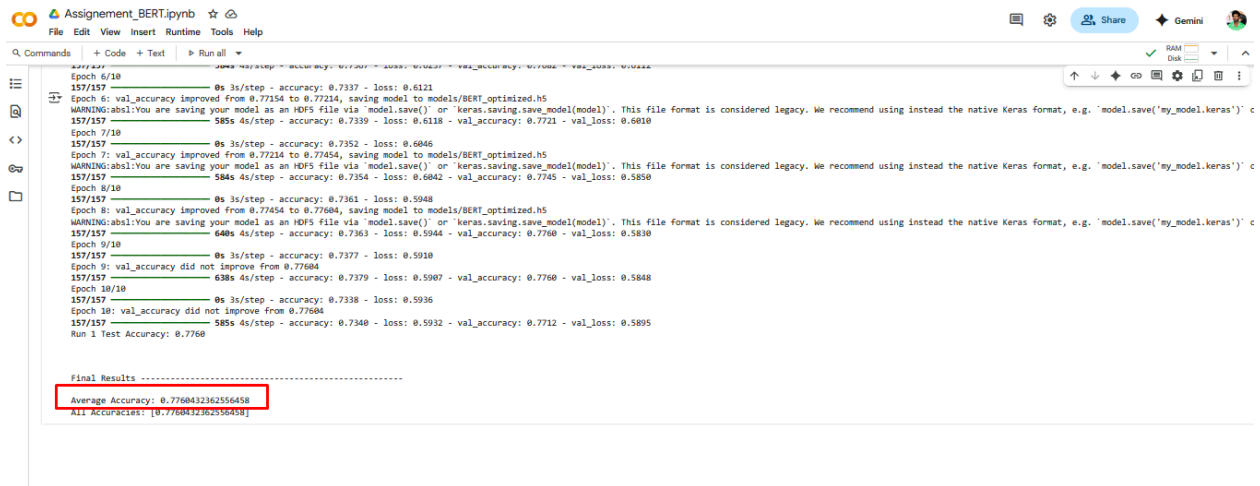


Figure 09: BERT accuracy for 10 epochs

- Accuracy and performance comparison**

Here are the accuracy and the performance comparison between each model above. Note that these are based on the above practical implementation.

Model/ feature	LSTM	LSTM (Advanced version)	Bidirectional LSTM	BERT
Accuracy	77.63% (averaged 5 epoch)	80.23% (averaged 5 epoch)	80.93% (averaged 5 epoch)	77.36% (5 epoch) 77.60% (10 epoch)
Training / Performance	Lowest training time	Took more time to training than LSTM, good performance	Took more time to training than LSTM advanced version, best performance	It took more time to than LSTM and difficult to train with lower time (took around 5-7 min for one epoch)
Complexity	Lowest	Low	Medium	High

- Conclusion**

The tweet data analysis shows that more complex models don't always perform better. The simple LSTM achieved 77.63% accuracy, while the advanced LSTM and Bidirectional LSTM gradually improved to 80.23% and 80.93% respectively. However, BERT, despite being the most sophisticated model, performed poorly at only 77.36-77.60% accuracy while taking much longer to train (5-7 minutes per epoch compared to much faster LSTM training times).

This pattern suggests that tweets are too short and simple for complex models like BERT to show their advantages. The Bidirectional LSTM worked best because it can read tweets both forwards and backwards to understand context, but isn't overly complicated like BERT. For tweet analysis, simpler models that focus on sequential word patterns seem to work better than heavy transformer models that are designed for longer, more complex texts.

Source codes

<https://github.com/MalithaDilshan/MSc/tree/main/AI/Lab%202>